James Hyle

CS450: Programming Language Translation

Dr. Sherman

5/3/26

Milestone 5: ANTLR Project - PeepHole Optimization

What didn't work:

This project went smoother than some of the previous checkpoints, but was still pretty tricky to figure out. The visitor pattern is still a bit confusing for me in this context where we are tracking multiple pieces of data as it goes through the optimization layer of code. Initially, I really struggled to understand how the visitors were interacting together and passing along instance variables. I did use A.I. to help me figure out what some of the visitor methods were doing, and to answer a few clarifying questions. I found that I didn't like the state machine model as much, and I found booleans to be a bit more readable for my usage. I am not sure why it is easier for me to think of flags in this context, but I did not like working with explicit states.

What did work:

I used the ASM plugin extensively in this checkpoint and it proved to be very useful in seeing the difference between the standard and optimized code. I identified portions of the code in each of the optimization files that I could focus on and found the pattern that the standard class writer was emitting, and used that to inform the optimizing classes of what instructions and data needed to be tracked. This checkpoint had a much different feel than the previous ones in that I only really spent time in the five optimization source files rather than taking on small chunks spread across many test cases. I think this checkpoint felt more manageable because of that aspect. I spent a lot of time researching the instructions to find out exactly how they are used.

https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html#jvms-6.5.iload This is excellent material I used, that was really useful in this assignment.

What I learned:

I am becoming more comfortable reading bytecode and understanding how the JVM is computing in larger programs. It is interesting to see how the computer is working at a relatively low level, and I have not used a stack based framework before so it is great practice in using different methods of computation. I think it is fascinating that underlying a large amount of source code is an even larger amount of bytecode that accomplishes the same semantics of the source code. I was constantly thinking about the time and space tradeoffs that optimizations use. We are able to get small gains by replacing an instruction or two, however it requires tracking the state of the program which uses space for what seems like a relatively small gain. Although I can imagine incremental gains have huge benefits for large scale applications, as we discussed in LLVM's BOLT framework. I am interested in finding a repository with more optimization patterns now that I am more familiar with them and the patterns needed to support them.